

AI大模型开发07深度学习—神经网络

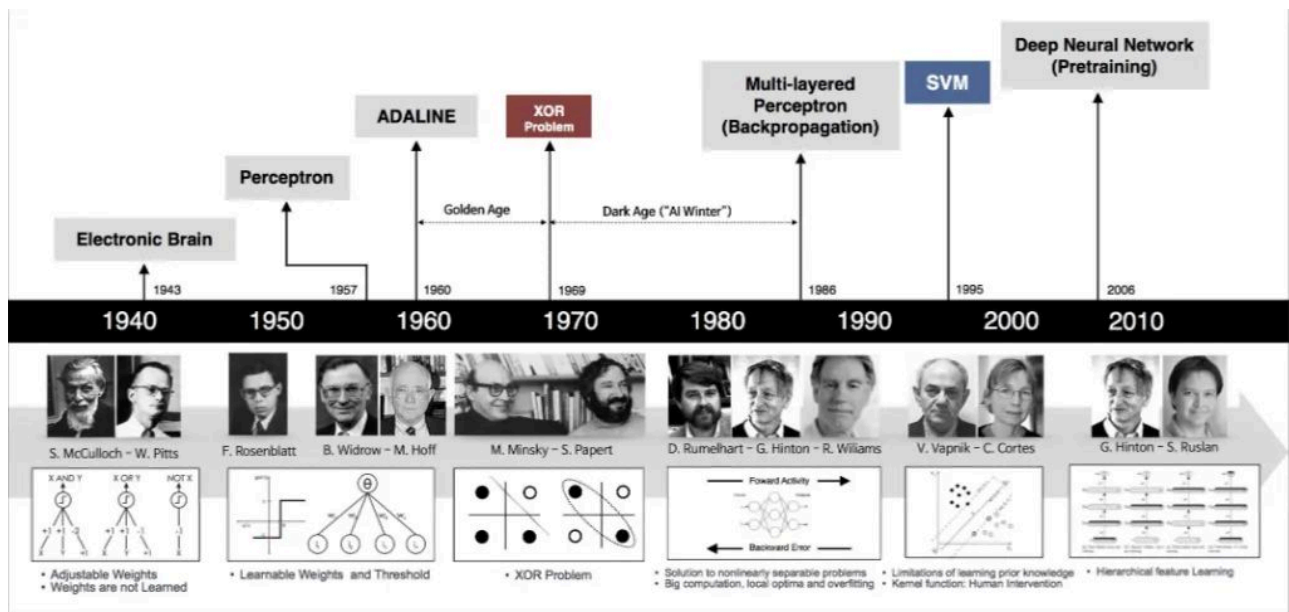
作者：AI芯源-邢朋辉

1.深度学习

1.1 深度学习概述

深度学习是人工智能技术取得突破性进展的关键推手，它根植于连接主义思想，即通过构建类人脑神经元连接的人工神经网络来学习并重现智能行为。其理论基础可追溯至上世纪40年代的神经网络研究，但得益于算力提升与大数据普及，才在近十年迎来爆发。

1.2 深度学习发展历程



人工智能的发展是一段从概念萌芽到技术爆发的壮阔历程：

1. 理论奠基（1950s）：艾伦·图灵提出“图灵测试”，为机器智能设立了哲学与工程学层面的思考起点。
2. 符号主义与早期探索（1950s-1980s）：人工智能研究早期以基于规则和逻辑的符号主义为主。
3. 机器学习兴起与第一次寒冬（1980s-1990s）：统计学习方法开始受到关注，但算力和数据限制导致进展缓慢，经历低谷。
4. 计算突破与专家系统（1997）：IBM“深蓝”计算机击败国际象棋世界冠军卡斯帕罗夫，展示了机器在复杂规则游戏中的超凡计算能力。
5. 数据驱动与深度学习复兴（2012）：杰弗里·辛顿团队凭借AlexNet在ImageNet图像识别大赛中以巨大优势夺冠，标志着基于深度卷积神经网络（CNN）的现代深度学习时代正式开启。

6. 感知智能突破（2016）：DeepMind的AlphaGo击败围棋世界冠军李世石，证明了深度学习与强化学习结合在超复杂策略博弈中的强大能力。
7. 大语言模型与生成式AI革命（2017-至今）：Transformer架构的提出（2017）彻底变革了自然语言处理。随后，GPT-3（2020）等千亿参数级大模型展现了惊人的内容生成与理解能力，推动AI进入“生成式”新阶段。

1.3 深度学习核心思想

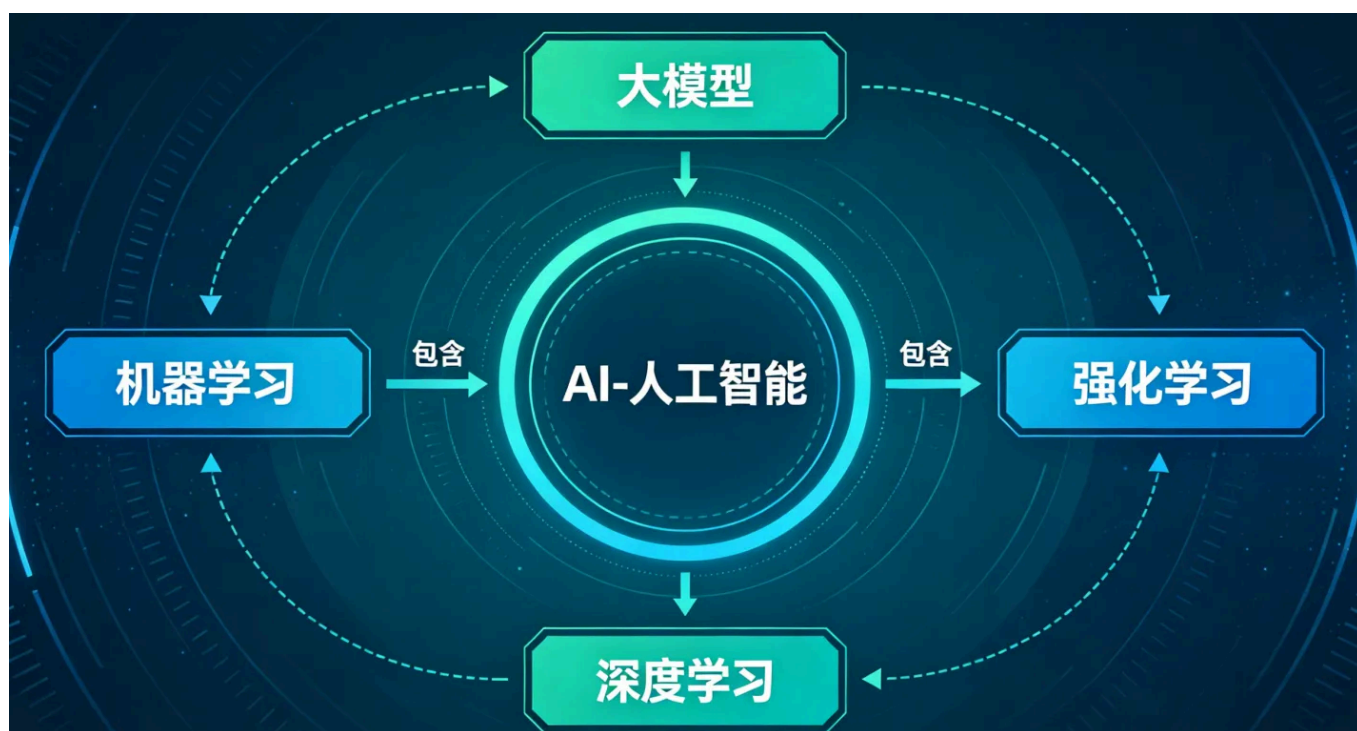
核心思想在于**分层特征学习**：通过多层非线性变换的神经网络，模型能够从原始数据（如像素、词汇）中自动逐层提取并组合出越来越抽象的高阶特征（如物体轮廓、语义概念），从而实现对复杂模式的识别与理解。

其强大的学习能力主要来源于以下三大支柱：

- **深度网络架构**：多层的非线性堆叠大幅提升了模型的表征能力，突破了传统浅层模型的性能瓶颈。
- **反向传播算法**：通过计算输出误差相对于网络权重的梯度，并逐层反向调整参数，使网络能在大规模数据中自动学习到数据的分布规律。
- **高效优化技术**：如随机梯度下降（SGD）及其变种（如Adam），通过迭代更新参数，高效地最小化损失函数，加速模型训练。

此外，理论研究表明，具有单隐藏层且足够多神经元的神经网络（配合适当的激活函数）可以逼近定义在紧集上的任何连续函数，这为神经网络的广泛应用提供了坚实的数学依据（即**通用近似定理**）。

1.4 深度学习VS机器学习



深度学习是机器学习的一个重要分支。

机器学习总体可分为三类，学习范式各有侧重：

- **监督学习**：算法从“带标签”的训练数据中学习输入与输出的映射关系，目标是找到能将输入映射到正确输出的模型。典型任务包括分类（识别图像类别）和回归（预测房价）。经典算法有线性回归、逻辑回归、支持向量机（SVM）和决策树。其核心是定义一个**损失函数**来衡量预测与真实标签的差距，并通过最小化损失来优化模型参数。
- **无监督学习**：处理“未标注”数据，目标是发现数据内在的结构、模式或分布。典型任务包括聚类（将相似数据分组，如客户分群）和降维（简化数据维度，如主成分分析PCA）。这类学习没有明确的优化目标，更多依赖数据本身的内在统计特性。
- **强化学习**：智能体在与环境的动态交互中学习最佳决策策略。其核心是**奖励机制**，智能体通过尝试不同动作，观察环境反馈的奖励或惩罚，学习出一套能最大化长期累积奖励的策略。著名案例如AlphaGo。算法包括Q-learning、策略梯度等。

机器学习模型可能面临**欠拟合**（模型太简单，学不到数据规律）与**过拟合**（模型太复杂，过分学到了训练数据的噪声）问题，需要通过正则化、验证集监控、调整模型复杂度等方法加以应对。

1.5 深度学习核心算法

模型	核心领域	解决的核心痛点	应用状态
CNN（卷积神经网络）	计算机视觉	图像空间特征提取	主流架构（如 ResNet、ViT 的前身）
RNN（循环神经网络）	序列建模	序列数据上下文依赖	基本被 LSTM/Transformer 替代
GAN（生成对抗网络）	生成任务	生成样本模糊、多样性不足	仍广泛用于图像生成、数据增强
DQN（深度 Q 网络）	强化学习	高维状态空间下的决策	基础方法，后续衍生出 PPO 等优化算法
GNN（图神经网络）	图数据建模	非欧几里得结构数据处理	快速发展的热门方向
LSTM（长短期记忆网络）	序列建模	RNN 长序列梯度消失	仍用于轻量级序列任务
Transformer	通用序列建模	长序列建模效率低、无法并行	大模型的核心架构

DBN（深度置信网络）	早期深度生成模型	深度网络训练难、易过拟合	作为历史模型，已被替代
-------------	----------	--------------	-------------

CNN：计算机视觉的基石

卷积神经网络专为处理图像、视频等网格状数据而设计。其核心在于**局部感受野**和**权值共享**：通过卷积核在图像上滑动，提取局部特征（如边缘、纹理）；同一卷积核在整个图像上共享参数，极大减少了计算量。之后再通过池化层进行下采样，保留主要特征，最后由全连接层完成分类等任务。这种结构使得CNN在图像分类、目标检测等任务上表现出色，并催生了ResNet、VGG等经典模型。

RNN：序列建模的先驱

循环神经网络旨在处理文本、语音、时间序列等具有先后顺序的数据。其最大特点是**循环连接**，即网络当前时刻的隐藏状态（记忆）由当前输入和**前一时刻的隐藏状态**共同决定，从而能够捕捉序列中的上下文依赖关系。正因如此，RNN一度是自然语言处理的主力。但其存在梯度消失/爆炸问题，难以有效处理长序列，因此其改进版——LSTM和GRU应运而生，通过门控机制更精细地控制信息留存与遗忘。

Transformer：大模型的核心架构

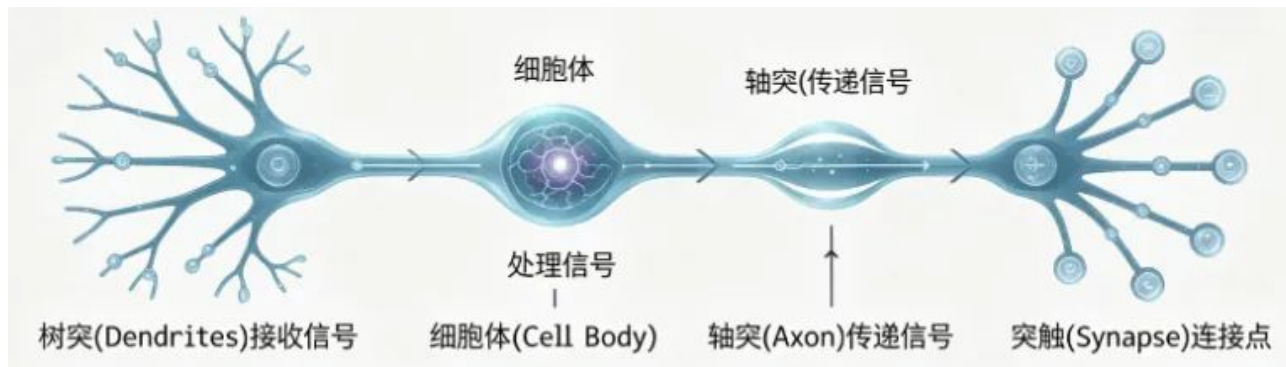
Transformer彻底摒弃了循环结构，完全基于**自注意力机制（Self-Attention）**。该机制允许模型在处理序列时，直接关注并计算序列中任意两个位置之间的关系，从而一次性地捕获**全局依赖**。这不仅解决了RNN无法并行计算的瓶颈，还极大提升了长距离依赖的建模能力。其编码器-解码器结构，结合**多头注意力**（从多个子空间并行提取不同层面的信息）、**位置编码**（为序列注入位置信息）和**残差连接**（稳定深层网络训练），构成了GPT、BERT等所有现代大语言模型的基石，实现了从自然语言处理到多模态理解的全面革新。

2.神经网络

2.1 神经网络

2.1.1 生物神经元

生物神经元结构：



神经网络：

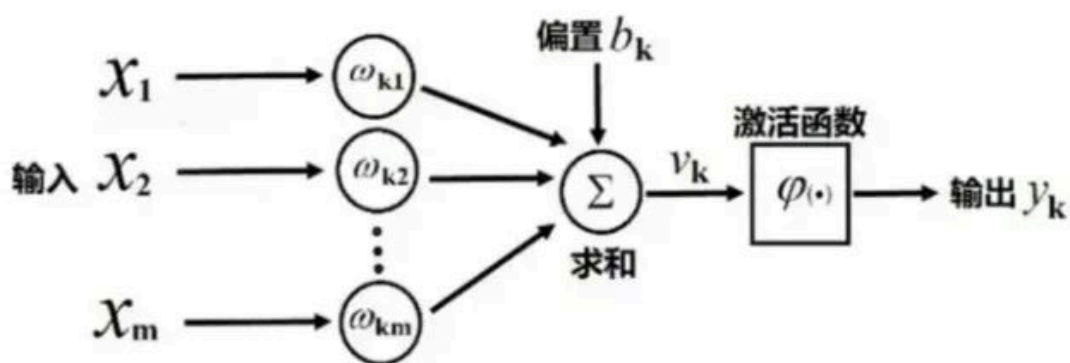
- **核心概念：**神经网络是受生物神经网络启发而建立的数学模型，由大量人工神经元相互连接构成。
- **起源：**人工神经元（或感知器）的基本元素最早由 **Frank Rosenblatt** 在 **20世纪50年代** 提出。
- **基本构成：**神经网络由许多并行运算、功能简单的神经元组成，神经元是构成神经网络的基本元素。

2.1.2 人工神经元

人工神经元是一个多输入/单输出的非线性器件，其数学模型是核心。

- **要素一：输入与加权：**
 - 输入信号 $x_1, x_2, \dots, x_n$ 来自其他神经元。
 - 每个输入对应一个连接权重 w_{ij} （表示从神经元 j 到神经元 i 的连接强度）。
 - **作用：**权重允许模型为不同输入分配不同的重要性。
- **要素二：阈值与线性组合：**
 - 神经元有一个阈值（或偏置） θ 。
 - 计算所有输入的加权和，加上偏置，得到净激活（net input）： $net = \sum_j w_j x_j + b$ ，其中 b 为偏置。
- **要素三：激活函数：**
 - 净激活值通过一个非线性激活函数 f 得到神经元输出： $y = f(net)$ 。
 - 若净激活 net 为正，神经元处于激活状态；若为负，则处于抑制状态。
- **向量化表示：**
 - 若将阈值视为一个输入 $x_0 = 1$ 的权重 w_{i0} ，则模型可简化为： $y = f(\mathbf{W}^T \mathbf{X})$ ，其中 $\mathbf{W}$ 为权重向量， $\mathbf{X}$ 为输入向量。

- **核心功能：**神经元（感知器）接收多个二进制输入，当加权总和超过激活电位（阈值）时，产生一个二进制输出，称为“放电”。
- **数学模型如下所示**



$$y_k = \varphi \left(\sum_{i=1}^m \omega_{ki} x_i + b_k \right) = \varphi (W_k^T X + b)$$

神经元的数学模型示意图

1. **输入加权求和：** $z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$
2. **激活函数：** $a = \sigma(z)$
3. **输出传递：** 将a传给下一层

关键对应关系：

- 树突 → 输入连接
- 权重 → 突触强度
- 细胞体 → 加权求和
- 激活函数 → 神经元是否激活
- 轴突 → 输出

2.1.3 应用领域

神经网络（包括其延伸的深度学习模型）理论支撑了广泛的应用，例如：

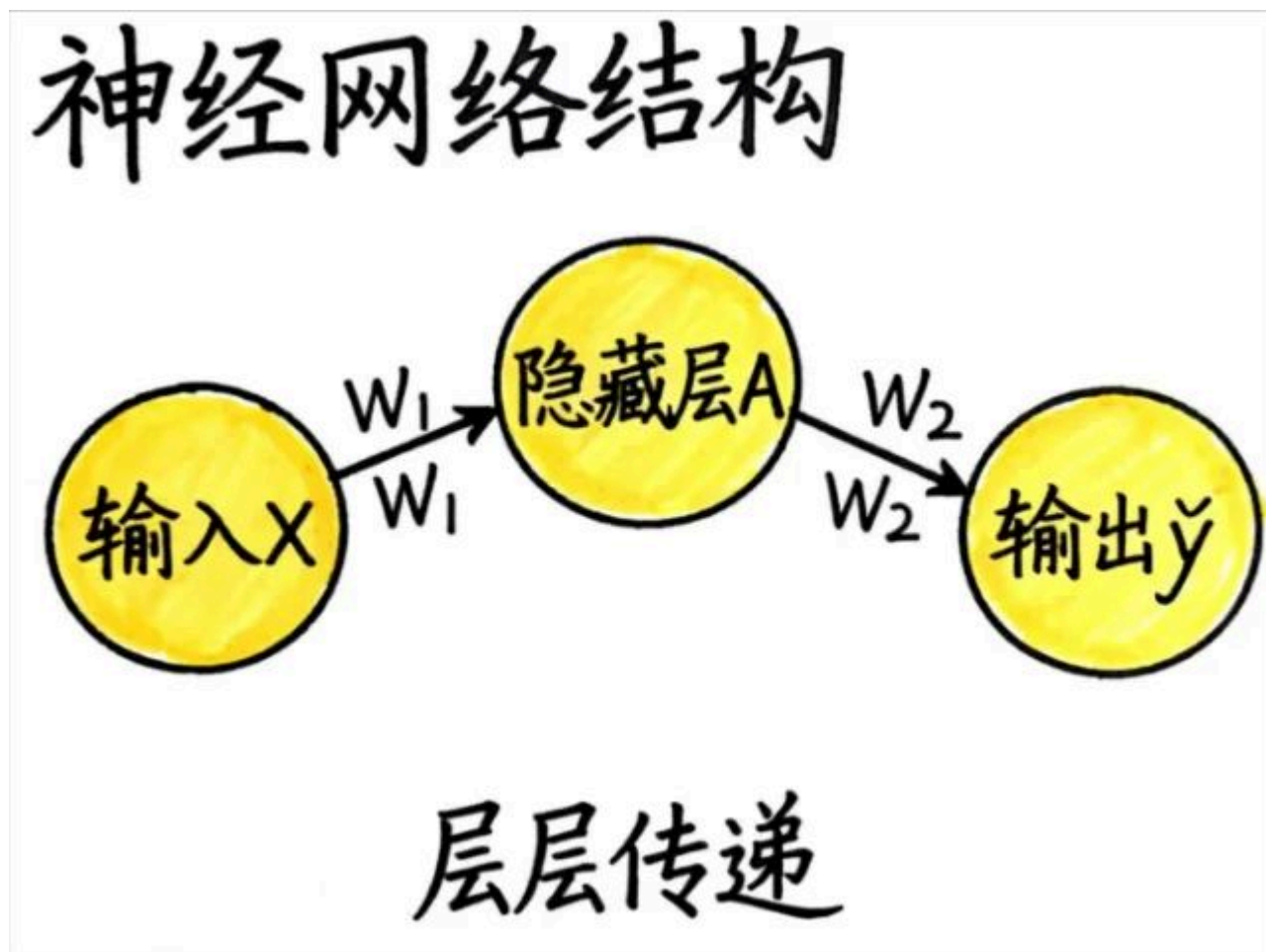
- **机器人控制**
- **图像处理**

- **大规模语言模型**：在复杂模型中，神经网络以多层形式组织。
 - PT-3 davinci版本包含 **96层** 神经网络。
 - DeepSeek R1为 **61层MoE + 前3层密集层**。
 - Kimi K2为 **61层**，其中 **1层为密集层**，其余 **60层为MoE层**。

2.1.4 神经网络组成

神经网络是深度学习的核心载体，其灵感源于生物神经系统。

一个典型的人工神经网络由以下部分组成：



- **输入层**：接收外部数据，每个节点对应一个输入特征（如图像的像素值）。
- **隐藏层**：位于输入与输出层之间，是网络的核心计算部分，负责对输入信息进行非线性变换和特征提取。层数越多，网络越“深”，学习能力越强。
- **输出层**：产生最终结果，如分类概率或回归数值。

举例：一个4层网络可能包含输入层 n 个神经元、第一个隐藏层 $n-1$ 个神经元、第二个隐藏层（2个神经元）、输出层 $n-2$ 个神经元。

- **连接权重**：每层节点与下一层节点间均有连接，每条连接附有一个可调节的**权重**，它决定了前一层输入对后一层的影响程度。

所有处理单元通过**连接权值**相互连接，构成一个有向图。

图数据结构（无向图、有向图）

权值集合是网络的长期记忆，可用权矩阵 $\mathbf{W}$ 表示。

权值类型分为**激发**（正值）和**抑制**（负值）。

- **状态空间**：每个神经元在时刻 t 有一个实数值状态（激励值），状态空间可以是连续的、离散的、有界的或无界的。常见取值包括二值（0/1 或 -1/1）或连续实数值。
- **偏置**：为每个节点添加的常量，用于提高模型的灵活性和拟合能力。
- **激活函数**：为网络引入非线性的关键，如Sigmoid、ReLU (Rectified Linear Unit) 或 Tanh 函数，它决定节点在收到加权输入后是否“激活”及如何输出。

其工作过程分为**前向传播**与**反向传播**：

- **前向传播**：输入数据从输入层经各层加权求和、叠加偏置并通过激活函数处理，逐层传递，最终在输出层得到预测结果。
- **反向传播**：计算预测与真实值之间的**损失**。利用链式法则，将损失从输出层向输入层逐层反向计算每个权重对总损失的“贡献度”（即梯度），然后通过优化算法（如梯度下降）按梯度方向微调所有权重和偏置，以逐步降低损失，使网络预测更准确。

2.1.5 前向传播

如果把神经网络比作一个“**黑盒流水线**”，那么**前向传播 (Forward Propagation)** 就是**产品从流水线一端进入，经过层层加工，最终出厂的过程**。

用一句话概括：**前向传播就是从“输入”计算“输出”的正向过程**。

举个例子：信息的正向流水线

想象你是一个刚入行的蛋糕师傅（神经元）：

1. **接收原料（输入 x ）**：顾客给了你面粉、鸡蛋（输入数据）。
2. **按配方加工（计算 $Wx+b$ ）**：你按照祖传秘方（权重 W ）按比例混合原料，并额外加了点糖（偏置 b ）。
3. **成品出炉（激活函数 σ ）**：放进烤箱，定型出炉，变成一个香喷喷的蛋糕（输出 y ）。

在神经网络中，这个过程是**单向的**，从第一层（输入层）一直流向最后一层（输出层），中间绝不复返。

2. 数学操作

抛开复杂的术语，前向传播在数学上干的就是最基础的**线性代数 + 简单函数变换**：

$$Z=W \cdot X+b$$

$$A=\sigma(Z)$$

(注： Z 是线性计算结果， A 是经过激活函数 σ 后的最终输出)

3. 前向传播的三大使命

在一个完整的神经网络训练中，前向传播主要负责：

- **传递信号**：把左边层的计算结果，原封不动地传给右边层。
- **特征提取**：通过每一层的权重 W ，把原始数据（比如一张猫的图片）慢慢转换成更高维的特征（比如识别出有耳朵、有胡须）。
- **算出预测值 (YY)**：走完整套流程，给出一个最终的答案（比如判断这是一只猫的概率为 90%）。

2.1.6 反向传播

负责根据预测误差，从输出到输入优化模型参数。

- **核心**：利用**链式法则**进行梯度反向传递。
- **流程**：
 - a. **计算损失梯度**：计算损失函数对输出层输出的梯度 $\frac{\partial \mathcal{L}}{\partial o}$ ，作为起点。
 - b. **梯度反向传递**：
 - 从输出层开始，逐层计算损失对模型参数的梯度。
 - 例如，对权重 $\mathbf{W}^{(2)}$ 的梯度： $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(2)}} = \frac{\partial \mathcal{L}}{\partial o} \cdot \mathbf{h}^\top$ ，其中 $\mathbf{h}$ 是隐藏层输出。
 - 梯度在每个节点由上一层传来的梯度与当前层激活函数的导数相乘得到。
 - c. **参数更新**：使用优化算法（如梯度下降）更新参数： $\mathbf{W} = \mathbf{W} - \eta \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{W}}$ ， η 为学习率。
 - d. **正则化处理**：若包含正则化项（如L2），需计算其对参数的梯度并加入更新。

实现：深度学习框架（如PyTorch、TensorFlow）通过构建**计算图**自动追踪计算过程，实现高效自动微分。

- **计算图**：一种可视化计算中运算符和变量依赖关系的方法，便于理解和实现反向传播。

前向传播：“从输入猜输出”（拿着当前的烂权重瞎猜，得出一个离谱的预测）。

反向传播：“从错误找原因”（拿着前向传播猜错的结果，反过来算算到底是哪里的权重拖了后腿，然后去修正它）。

2.1.7 神经网络核心算法

顺序	算法/概念	核心作用	关联知识
1	感知机 (Perceptron)	理解线性分类的最简单单元。	等同于逻辑回归的神经元版本。
2	多层感知机 (MLP)	解决非线性问题（万能近似定理）。	机器学习中的神经网络，只是层数更深。
3	激活函数	引入非线性（ReLU, Sigmoid, Tanh）。	类似于特征变换，但发生在层与层之间。
4	反向传播 (Backpropagation)	灵魂算法。计算梯度，更新参数。	梯度下降的链式求导版。
5	优化器 (Optimizer)	控制参数更新的方式和速度。	SGD → Momentum → Adam（必会）。
6	正则化 (Regularization)	防止过拟合。	Dropout（随机失活）、L1/L2、Early Stopping。

2.2 损失函数

损失函数（Loss Function）是神经网络训练的核心组件，用于量化模型预测值与真实标签之间的差距。优化的目标即是通过调整模型参数（权重和偏置） θ ，使该损失值最小化，从而提升模型的预测精度。

2.2.1 定义与核心作用

损失函数提供了一个可微分的标量度量，以指导模型优化方向。假设一个神经网络模型用参数 θ 做出预测 $\hat{y}$ ，而真实值为 y ，则损失函数 $L(y, \hat{y})$ 评估了此次预测的“错误”程度。更小的损失值意味着模型预测更接近真实值。损失函数是连接前向传播与反向传播的关键纽带。

2.2.2 数学原理与核心形式

定义在含有 n 个样本的数据集上，多数损失函数最终形式都是对单个样本损失的平均。

其通用目标为：

$$\min_{\theta} \frac{1}{n} \sum_{i=1}^n L(y_i, \hat{y}_i)$$

根据任务类型（回归或分类），损失函数的数学构造原理与具体形式迥异。

均方误差（MSE）与 交叉熵（Cross-Entropy）

分别是回归与分类任务的两大基石性损失函数。

均方误差（Mean Squared Error, MSE）

数学形式：

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

核心作用与数学原理：

MSE 是单点损失 $(y - \hat{y})^2$ 的均值。其平方项使得函数处处可导，导数为 $2(y - \hat{y})$ ，这为回归任务中的梯度计算提供了线性且直接的路径。

特点：因其平方项，对预测中的大误差（离群值）惩罚力度极大，这也使其对异常值敏感。

交叉熵损失（Cross-Entropy Loss）

数学形式（二分类）：

$L_{BCE} = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$ 其中 $\hat{y}_i$ 是模型预测的样本 i 为正类的概率。

核心作用与数学原理：

交叉熵源自信息论，衡量两个概率分布 P （真实分布，由其标签 y 决定）与 Q （预测分布 $\hat{y}$ ）的差异。当 $\hat{y}_i$ 接近真实标签 y_i 时，损失 L 值最小。

特点：特别适用于分类任务，最大化模型预测的“对数似然”，梯度形式 $\frac{\partial L}{\partial \hat{y}} = \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})}$ 具有优良的特性，能避免梯度饱和问题。

误差项 $\delta^{(o)}$ 的推导

以 Sigmoid 激活函数 σ 为例，输出层线性组合为 $z^{(o)}$ ，预测值 $\hat{y} = \sigma(z^{(o)})$ 。损失 L 对 $z^{(o)}$ 的梯度（即误差项 $\delta^{(o)}$ ）为：

$$\delta^{(o)} = \frac{\partial L}{\partial z^{(o)}} = \underbrace{\frac{\partial L}{\partial \hat{y}}}_{\text{损失梯度}} \cdot \underbrace{\frac{\partial \hat{y}}{\partial z^{(o)}}}_{\text{激活导数}}$$

代入具体公式可得：

$$\delta^{(o)} = (\hat{y} - y) \odot \sigma'(z^{(o)})$$

2.2.3 代码实现

可以直接实现还可以通过numpy实现还可以通过框架Pytorch实现

在 PyTorch 中，损失函数封装于 `torch.nn` 模块，使用极为便利。

[PyTorch 中文文档](#)

1. MSE实现

Python

```
1 import numpy as np
2
3 def mse_loss_numpy(y_true, y_pred):
4     """
5     使用 NumPy 计算均方误差
6     """
7     # 将输入转换为 numpy 数组以确保计算正常
8     y_true = np.array(y_true)
9     y_pred = np.array(y_pred)
10
11     # 核心计算: (差值 -> 平方 -> 求平均)
12     # 这行代码对应公式:  $1/N * \sum((y\_pred - y\_true)^2)$ 
13     loss = np.mean((y_pred - y_true) ** 2)
14
15     return loss
16
17 # 测试
18 y_true = np.array([0, 1, 1, 0])
19 y_pred = np.array([0.1, 0.9, 0.8, 0.2])
20 loss = mse_loss_numpy(y_true, y_pred)
21 print(f"NumPy 计算的 MSE 损失: {loss:.4f}") # 输出 0.0250
```

2. 二分类交叉熵实现

基于Numpy实现

Python

```
1 import numpy as np
2
3 def binary_cross_entropy_numpy(y_true, y_pred):
4     """
5     NumPy 向量化计算 BCE
6     """
7     y_true = np.array(y_true, dtype=np.float64)
8     y_pred = np.array(y_pred, dtype=np.float64)
```

```
9
10     # 数值稳定性处理：裁剪预测值，防止 log(0)
11     eps = 1e-15
12     y_pred = np.clip(y_pred, eps, 1 - eps)
13
14     # 核心公式
15     loss = -np.mean(y_true * np.log(y_pred) + (1 - y_true) * np.log(1 -
y_pred))
16
17     return loss
18
19 # 测试
20 y_true = np.array([0, 1, 1, 0])
21 y_pred = np.array([0.1, 0.9, 0.8, 0.2])
22 loss = binary_cross_entropy_numpy(y_true, y_pred)
23 print(f"NumPy 计算的 BCE 损失: {loss:.4f}")
```

2.2.4 损失函数类型与应用场景

任务大类	子任务/数据特性	推荐损失函数	核心原理与适用逻辑
------	----------	--------	-----------

回归任务 (预测连续数值--数字)	通用场景（预测房价、温度）	均方误差 (MSE)	计算误差平方的均值。作为最常用的回归损失，它对大误差惩罚更重，引导模型快速收敛，但对离群值敏感。
	数据包含较多异常值	平均绝对误差 (MAE)	计算误差绝对值。对异常值的惩罚是线性的，鲁棒性强，避免了被极端值过度影响的问题。
	兼顾平滑性与鲁棒性	Huber Loss	由超参数 δ 控制：误差 e 小于 δ 时按 MSE 处理以保证可导性；大于 δ 时按 MAE 处理以保证稳定性。

分类任务 (预测离散类别)	二分类问题	交叉熵 (Cross-Entropy, CE)	衡量预测概率分布与真实标签分布的差异。是分类任务的标准损失，能引导模型为正确类别赋予高置信度，梯度特性优。
	多分类问题（互斥）	交叉熵 (CE) 配合 Softmax	对每个样本强制概率之和为 1，并最大化正确类别的概率。Sparse 变体可直接接收类别索引，无需进行 One-hot 编码。
	类别极度不平衡	加权交叉熵 / Focal Loss	加权交叉熵通过调整各类别权重，强迫模型关注少数类。Focal Loss 则动态降低简单样本的损失权重，聚焦难分的关键样本。

生成模型 (AI作画、内容生成)	生成对抗网络 (GANs)	对抗损失 (Adversarial Loss)	生成器与判别器进行零和博弈。判别器损失评估其对真假样本的区分能力；生成器损失则试图让判别器误判，以此驱动生成更逼真的数据。
	变分自编码器 (VAEs)	证据下界 (ELBO)	ELBO = 重构损失（如 MSE）+ KL 散度。前者确保输出还原输入，后者约束潜在空间的分布接近先验（如标准正态），并具有可解释性。

2.2.5 损失函数选择策略

选择最佳损失函数是模型成功的关键步骤。可以从以下几个维度考量：

1. 任务类型决定基本方向：回归任务优先考虑 MSE、MAE 或其变体；分类任务首选交叉熵及其变体。
1. 数据特性决定变体选择：
 - 异常值敏感度：若数据噪音大、离群点多，MSE因平方项而放大误差可能导致模型不稳定，此时应选用 MAE 或 Huber Loss。
 - 类别不平衡：在多分类任务中，若某类别样本极少，可在交叉熵中使用 weight 参数，或采用 Focal Loss，给予稀有类别更大惩罚权重。
2. 模型性能与优化难度：交叉熵因其处处平滑的梯度特性，在深度网络分类训练中通常比平方误差更高效、收敛更快。
2. 结合正则化：为防止过拟合，实际工程中往往在损失函数中引入正则化项，形成最终优化目标：

$L_{total} = L_{data}(y, \hat{y}) + \lambda \cdot R(\theta)$ 其中 $R(\theta)$ 可以是 $L1$ 或 $L2$ 范数， λ 为正则化系数。

2.2.6 结合优化器与梯度下降

损失函数本身不包含优化，必须与优化器（如梯度下降）结合使用。以下以线性模型 $\hat{y} = w \cdot x + b$ 为例，演示如何计算 MSE 损失并用梯度下降法更新参数。

1. 前向传播:

$$\hat{y} = w \cdot x + b$$

2. 计算损失 (MSE):

$$L = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

3. 梯度计算 (反向传播):

利用 MSE 的导数，计算参数 w 和 b 的梯度:

$$\frac{\partial L}{\partial w} = \frac{1}{n} \sum_{i=1}^n 2(\hat{y}_i - y_i) \cdot x_i \quad \frac{\partial L}{\partial b} = \frac{1}{n} \sum_{i=1}^n 2(\hat{y}_i - y_i)$$

4. 参数更新 (梯度下降):

设学习率为 η ，参数更新公式为:

$$w_{\text{new}} = w_{\text{old}} - \eta \cdot \frac{\partial L}{\partial w} \quad b_{\text{new}} = b_{\text{old}} - \eta \cdot \frac{\partial L}{\partial b}$$

在高级框架中，此流程被抽象为**损失计算**、**自动微分**、**优化器更新**三个步骤:

Python

```
1 import numpy as np
2
3 # 生成模拟数据
4 x_data = np.array([[1.0], [2.0], [3.0]])
5 y_data = np.array([[2.0], [4.0], [6.0]])
6
7 # 初始化参数
8 w = np.random.randn(1)
9 b = np.random.randn(1)
10 learning_rate = 0.01
11 num_iterations = 100
12
13 for i in range(num_iterations):
14     # 前向传播
15     y_pred = w * x_data + b
16
17     # 计算损失
18     loss = np.mean((y_pred - y_data) ** 2)
19     if i % 20 == 0:
20         print(f'Iteration {i}, Loss: {loss:.4f}, w: {w[0]:.4f}, b: {b[0]:.4f}')
```

```
21
22     # 梯度计算
23     grad_w = np.mean(2 * (y_pred - y_data) * x_data)
24     grad_b = np.mean(2 * (y_pred - y_data))
25
26     # 参数更新
27     w = w - learning_rate * grad_w
28     b = b - learning_rate * grad_b
29
30 print(f'Training finished, final w: {w[0]:.4f}, b: {b[0]:.4f}')
```

注意：损失函数数值是监控模型训练的指标；而真正的优化动力来自损失对参数的梯度。在深度学习框架（PyTorch/TensorFlow）中，链式求导（反向传播）由自动微分系统接管，极大简化了工程复杂度。

3.核心算法

3.1 感知机

3.1.1 感知机

感知机（Perceptron），就是人工智能发展史上**第一台真正意义上的“自动纠错机”**。

它是所有神经网络的祖师爷，Support Vector Machine (SVM) 的灵感源泉。

从生物学的角度来看，人类之所以能思考、能感知，是因为大脑中有数以亿计的**神经元（Neuron）**在相互传递信号。1957年，心理学家 Rosenblatt 灵光一闪：**能不能用数学公式模拟一个生物神经元？**

于是，**感知机**诞生了。它的结构极其简单，分为三步：

1. **收件箱（输入层）**：接收外界传来的各种信号 $x_1, x_2, \dots, x_n$ （比如：颜值、性格、车房情况）。
2. **加工坊（加权求和）**：给每个信号加上一个重要性权重 $w_1, w_2, \dots, w_n$ ，然后统统加起来，再减去一个门槛（偏置 b ）。
3. **发令枪（激活函数）**：看看加总的结果是不是大于0。如果大于0，就 firing（输出1，代表喜欢/是正类）；否则就不 firing（输出0，代表不喜欢/是负类）。

感知机本质上就是在多维空间里**找一张纸（超平面）**，把这堆数据生生切成两半，一半标为1，一半标为0。

3.1.2 感知机的训练

一个刚出生的婴儿分不清猫和狗，感知机一开始也是一个“纯纯的傻瓜”，它的初始权重全是随机乱设的。那它是怎么通过“上课”变聪明的呢？

这就回到了我们上节课聊的核心——**损失函数 (Loss Function)**。

感知机的训练过程，就是一个“**发现错误 → 纠正错误**”的死循环：

1. **前向传播**：拿一条数据进来，感知机根据当前的权重算出一个预测结果。
2. **比对答案**：如果预测对了，皆大欢喜，继续下一条。
3. **碰上错题（感知机更新的灵魂）**：如果**真实是1**，模型却算了**0**（漏报）：说明门槛设太高了。感知机会把权重 w 往大了调，把偏置 b 往小了调，让下次更容易 firing。如果**真实是0**，模型却算了**1**（误报）：说明门槛设太低了。感知机会把权重 w 往小了调，把偏置 b 往大了调，让下次更难 firing。
4. **迭代**：反复把所有数据过很多遍 (Epoch)，直到模型不再犯错，或者实在改不动了为止。

3.1.3 感知机的宿命

1956年，AI发展史上的“里程碑事件”发生了——**达特茅斯会议**。在美国达特茅斯学院，10位科学家召开了一场为期两个月的研讨会，会上首次提出了“**人工智能 (Artificial Intelligence)**”这一术语，明确了AI的7大研究目标：**自然语言处理、自动计算机、神经网络、抽象概念形成、机器学习、创造力、美学感知**。

在1969年，人工智能界发生了一件轰动一时的大事件，被称为“AI的第一个寒冬”。这一切，都源于感知机的一个**致命缺陷**。

如果在二维平面上，你能画一条直线把两类数据完美分开（比如：男生站左边，女生站右边），感知机绝对能秒秒钟帮你算出这条直线的公式。它 **Guarantee（保证）** 能在有限步内收敛。

滑铁卢：死活搞不定“异或” (XOR)

但如果遇到**非线性可分**的问题，感知机就彻底抓瞎了。

最经典的例子就是**异或问题 (XOR)**：

- 输入 (0, 0) -> 输出 0
- 输入 (1, 1) -> 输出 0
- 输入 (0, 1) -> 输出 1
- 输入 (1, 0) -> 输出 1

你无论怎么画图，都不可能用**一条直线**把 (0,1) 和 (1,0) 与另外两点分开。面对这种情况，感知机只会像个倔驴一样在原地打转，永远找不到解。

当时，AI领域的巨头 Minsky 出版了《Perceptrons》一书，狠狠戳穿了这个痛点。学术界大失所望，纷纷撤资，导致了AI领域的第一次大萧条

感知机的遗产

虽然感知机因为过于简单而经历了滑铁卢，但它留下的思想火种却点燃了后世的两大主流流派：

1. **催生了 SVM（支持向量机）**：既然一张纸切不开，那我就找一张**最厚**的纸（最大间隔超平面），并且引入核函数把数据映射到更高维度去切。
2. **催生了 深度学习/神经网络**：既然一层切不开，那我就**堆叠多层感知机（MLP）**！这就是后来风靡全球的**多层感知机（Multilayer Perceptron, MLP）**和**反向传播算法（Backpropagation）**的由来。

感知机带来的“**加权求和 → 非线性激活 → 计算误差 → 梯度下降更新**”这一套至今仍在沿用的机器学习核心流水线。

神经网络的“单细胞”版本，也是最早的AI模型之一。它是一个**线性二分类模型**。你可以把它想象成一个只有输入和输出的极简电路开关。

3.1.4 感知机模型

$$f(x) = \text{sign}(w^T x + b)$$

其中：

- $x = [x_1, x_2, \dots, x_n]^T$ 是输入向量
- $w = [w_1, w_2, \dots, w_n]^T$ 是权重向量
- b 是偏置项
- $\text{sign}()$ 是符号函数

几何解释：

- $w^T x + b = 0$ 定义了一个超平面（决策边界）
- 感知机将空间分为两个半空间
- 正类： $w^T x + b > 0$
- 负类： $w^T x + b < 0$

学习规则：

当预测错误时，更新规则为：

$$w_{new} = w_{old} + \eta(y - \hat{y})x$$

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

其中 η 是学习率

收敛定理：

如果数据线性可分，感知机算法保证在有限步内收敛到一个解。

局限性：

1. 只能解决线性可分问题
2. 无法处理XOR等非线性问题
3. 无法给出概率输出

实现过程

1. **加权求和**：接收多个输入信号 ($x_1, x_2, \dots$)，每个信号乘以对应的权重 ($w_1, w_2, \dots$)，再加上一个偏置 (b)。
2. **激活决策**：将求和结果送入一个**阶跃函数** (Step Function)。如果结果大于0，输出1 (正类)；否则输出0 (负类)。

3.1.5 代码实现

就像你决定是否去打球。有三个因素：天气(x_1)、心情(x_2)、朋友去不去(x_3)。每个因素对你影响程度不同 (权重 w)。你把这三个因素加起来，如果总分超过你的心理底线 (阈值)，你就去；否则就不去。

Python

```
1 import numpy as np
2
3 class Perceptron:
4     def __init__(self, learning_rate=0.1, n_iters=100):
5         """
6         初始化感知机
7         :param learning_rate: 学习率 (步长)，控制权重更新的幅度
8         :param n_iters: 遍历整个数据集的最大次数 (Epochs)
9         """
10        self.lr = learning_rate
11        self.n_iters = n_iters
12        self.weights = None # 权重向量
13        self.bias = None    # 偏置项 (Bias)
14
15    def fit(self, X, y):
16        """
```

```

17 训练模型：寻找最优的超平面
18 :param X: 特征矩阵 [n_samples, n_features]
19 :param y: 标签向量 [n_samples], 这里我们假设标签是 0 和 1
20 """
21 n_samples, n_features = X.shape
22
23 # 1. 初始化参数：权重全为0（或小的随机数），偏置为0
24 # 注意：为了简化，这里初始化为0。在实际应用中，有时会初始化为小的随机数。
25 self.weights = np.zeros(n_features)
26 self.bias = 0
27
28 # 2. 感知机要求标签是 1 和 -1，所以我们把 0 转换成 -1
29 # 这是为了方便计算：如果预测错了，更新公式是  $w += lr * y * x$ 
30 y_ = np.array([1 if i > 0 else -1 for i in y])
31
32 # 3. 开始训练循环
33 for _ in range(self.n_iters):
34     errors = 0
35     # 遍历每一个样本
36     for idx, x_i in enumerate(X):
37         # 线性组合:  $z = w \cdot x + b$ 
38         linear_output = np.dot(x_i, self.weights) + self.bias
39
40         # 阶跃函数 (Step Function) 激活：大于0为1，否则为-1
41         # 这里我们用 sign 函数
42         y_predicted = 1 if linear_output >= 0 else -1
43
44         # 4. 检查是否预测错误
45         # 如果 y_ 和 y_predicted 符号不同 (相乘 < 0)，说明错了
46         if y_[idx] * y_predicted <= 0:
47             errors += 1
48
49             # 5. 更新规则 (核心!)
50             # 如果是正样本被判为负，就加大权重；反之则减小
51             update = self.lr * y_[idx]
52             self.weights += update * x_i
53             self.bias += update
54
55         # 如果没有错误，说明已经收敛，提前停止
56         if errors == 0:
57             print(f"模型在第 {_ + 1} 轮收敛!")
58             break
59     return self
60
61 def predict(self, X):
62     """
63     预测新数据
64     """
65     linear_output = np.dot(X, self.weights) + self.bias

```

```

65         # 转换回 0/1 标签
66         return np.where(linear_output >= 0, 1, 0)
67
68 # --- 测试部分 ---
69 if __name__ == "__main__":
70     # 1. 准备数据：与非门 (NAND) 数据
71     # 我们知道 NAND 门是线性可分的，感知机可以搞定
72     # 输入：(0,0), (0,1), (1,0), (1,1)
73     # 输出：1, 1, 1, 0
74     X = np.array([[0, 0],
75                   [0, 1],
76                   [1, 0],
77                   [1, 1]])
78
79     y = np.array([1, 1, 1, 0]) # 注意这里是 0 和 1
80
81     # 2. 创建并训练感知机
82     p = Perceptron(learning_rate=0.1, n_iters=10)
83     p.fit(X, y)
84
85     # 3. 查看训练后的参数
86     print(f"训练后的权重 (Weights): {p.weights}")
87     print(f"训练后的偏置 (Bias): {p.bias}")
88
89     # 4. 测试预测
90     test_input = np.array([[0, 0], [1, 1]])
91     predictions = p.predict(test_input)
92     print(f"预测结果 (0,0 和 1,1): {predictions}") # 应该输出 [1, 0]

```

3.2 多层感知机 (MLP)

3.2.1 多层感知机

感知机是“单层”的线性分类器，那么**多层感知机 (Multilayer Perceptron, MLP)**就是给它“叠buff”。

为了解决感知机无法处理**非线性问题 (如异或 XOR)**的致命缺陷，科学家们在输入层和输出层之间加了一个或多个**隐藏层 (Hidden Layer)**，并引入了**非线性激活函数**。

这一改动，直接宣告了**深度学习 (Deep Learning)**的诞生。

3.2.2 为什么单层感知机搞不定 XOR?

感知机“原地打转”的 XOR 问题

- **输入：** $(0,0) \rightarrow 0, (1,1) \rightarrow 0, (0,1) \rightarrow 1, (1,0) \rightarrow 1$ 。

- **困境**：在二维平面上，你**永远无法用一条直线**把 (0,1) 和 (1,0) 这两点，与 (0,0) 和 (1,1) 这两点完全切开。

MLP 的解题思路：既然一条直线切不开，就用**两条直线**（两个神经元）分别切一刀，然后在最后汇总的时候，把这两个结果“与”起来。

这就是**隐藏层**的作用——**特征变换**。

感知机的升级版，也叫**全连接神经网络**。它由输入层、隐藏层（至少一层）和输出层组成。关键在于引入了**非线性**，让模型能够处理像异或（XOR）这样复杂的问题。

3.2.3 MLP 的核心架构

一个标准的 MLP 通常包含三层结构：

1. **输入层 (Input Layer)**：接收原始数据（如 XOR 的两个输入 x_1, x_2 ）。
2. **隐藏层 (Hidden Layer)**：这是 MLP 的灵魂。它不直接对接输出，而是对输入进行复杂的非线性变换。每一层都有若干神经元，每个神经元都包含“加权求和”和“激活函数”。
3. **输出层 (Output Layer)**：根据隐藏层提取的特征，给出最终的预测结果。

关键点：如果只有层数的堆叠，而没有**非线性激活函数**（如 ReLU, Sigmoid），那么无论叠多少层，整个网络在数学上还是等价于一个线性变换，依然解不了 XOR。

3.2.4 实现过程

前向传播：数据从输入层进入，经过第一层神经元的加权求和与激活，传给下一层，直到输出层。

层级堆叠：每一层的神经元都与上一层的所有神经元相连（全连接）。

通用近似定理

定理 (Cybenko, 1989)：

一个前馈神经网络，如果具有：

1. 至少一个隐藏层
2. 足够多的神经元
3. 非线性激活函数

则可以以任意精度逼近任何连续函数。

数学表达：

对于任意连续函数 $f : [0, 1]^n \rightarrow \mathbb{R}$ 和任意 $\epsilon > 0$ ，存在一个单隐藏层神经网络：

$$F(\mathbf{x}) = \sum_{i=1}^N v_i \sigma(\mathbf{w}_i^\top \mathbf{x} + b_i)$$

使得：

$$|F(\mathbf{x}) - f(\mathbf{x})| < \varepsilon, \quad \forall \mathbf{x} \in [0, 1]^n$$

其中 σ 是一个非多项式、有界、单调递增的连续激活函数（如 Sigmoid）。

直观理解：

- 神经网络可以构造任意复杂的决策边界。
- 每个隐藏层神经元可以看作一个“基函数”，对输入空间进行非线性变换。
- 多个这样的基函数的线性组合（通过输出层权重 v_i ）可以逼近任意连续函数。

这个定理从理论上保证了，只要网络足够宽（隐藏层神经元足够多），即使只有一层隐藏层，也能具备强大的函数拟合能力。不过在实践中，深度（更多隐藏层）往往比宽度（更多神经元）更高效、更利于学习复杂特征。

原生实现：

感知机是个单细胞生物，只能做直线分割。MLP 就是一个**大脑团队**。第一层专家（隐藏层1）看原始数据，提取简单特征（比如线条）；第二层专家（隐藏层2）看第一层的报告，组合成复杂特征（比如眼睛）；最后由老板（输出层）拍板定论。

Python

```

1 import numpy as np
2
3
4 class MLP:
5     def __init__(self, input_size=2, hidden_size=2, output_size=1,
6         lr=0.5):
7         # 1. 初始化参数（权重和偏置）
8         # w1: 输入层 -> 隐藏层
9         # w2: 隐藏层 -> 输出层
10        self.W1 = np.random.randn(input_size, hidden_size)
11        self.b1 = np.zeros((1, hidden_size))
12        self.W2 = np.random.randn(hidden_size, output_size)
13        self.b2 = np.zeros((1, output_size))
14        self.lr = lr
15
16    def sigmoid(self, x):
17        # 激活函数：引入非线性
18        return 1 / (1 + np.exp(-x))
19
20    def sigmoid_derivative(self, x):
```

```

20     # Sigmoid的导数，用于反向传播
21     return x * (1 - x)
22
23 def train(self, X, y, epochs=10000):
24     for epoch in range(epochs):
25         # --- 前向传播 (Forward Propagation) ---
26         # 隐藏层计算
27         z1 = np.dot(X, self.W1) + self.b1 # 线性变换
28         a1 = self.sigmoid(z1) # 非线性激活
29
30         # 输出层计算
31         z2 = np.dot(a1, self.W2) + self.b2
32         a2 = self.sigmoid(z2) # 最终预测值 (0~1之间)
33
34         # --- 反向传播 (Backpropagation) ---
35         # 1. 计算误差 (Loss Derivative)
36         # 这里用的是均方误差(MSE)的导数简化版：(预测值 - 真实值)
37         output_error = y - a2
38
39         # 2. 输出层梯度
40         # d_a2 = output_error * sigmoid_derivative(a2)
41         d_a2 = output_error * self.sigmoid_derivative(a2)
42
43         # 3. 隐藏层梯度 (链式法则)
44         # error 传回隐藏层
45         d_a1 = np.dot(d_a2, self.W2.T) * self.sigmoid_derivative(a1)
46         # 4. 更新权重和偏置 (Gradient Descent)
47         self.W2 += np.dot(a1.T, d_a2) * self.lr
48         self.b2 += np.sum(d_a2, axis=0, keepdims=True) * self.lr
49         self.W1 += np.dot(X.T, d_a1) * self.lr
50         self.b1 += np.sum(d_a1, axis=0, keepdims=True) * self.lr
51
52         # 打印训练进度
53         if epoch % 2000 == 0:
54             loss = np.mean(np.square(y - a2))
55             print(f"Epoch {epoch}, Loss: {loss:.6f}")
56
57 def predict(self, X):
58     z1 = np.dot(X, self.W1) + self.b1
59     a1 = self.sigmoid(z1)
60     z2 = np.dot(a1, self.W2) + self.b2
61     a2 = self.sigmoid(z2)
62     # 将概率转为 0 或 1
63     return (a2 > 0.5).astype(int)
64
65
66 # --- 测试：解决 XOR 问题 ---
67 if __name__ == "__main__":

```

```

68     # XOR 数据
69     X = np.array([[0, 0],
70                  [0, 1],
71                  [1, 0],
72                  [1, 1]])
73     y = np.array([[0], [1], [1], [0]])
74
75     print("开始训练多层感知机 (MLP)...")
76     mlp = MLP(lr=0.5)
77     mlp.train(X, y, epochs=10000)
78
79     print("\n训练完成, 预测结果: ")
80     predictions = mlp.predict(X)
81     print(predictions)

```

MLP 的强大之处在于**反向传播 (Backpropagation)**:

1. **前向传播 (Forward Pass)**: 数据从输入进去, 经过隐藏层的非线性变换, 吐出一个结果。
2. **计算误差 (Loss)**: 看这个结果和真实答案差多远。
3. **反向传播 (Backward Pass)**: 这是最精妙的一步。误差并不直接修改输出层, 而是通过**链式法则 (Chain Rule)**, 把误差“分摊”给隐藏层的每一个神经元。
隐藏层的神经元会收到信号: “嘿, 你刚才的输出导致了最后的错误, 你得改改你的权重。”
4. **迭代**: 通过成千上万次的“试错-反馈-修正”, 隐藏层自动学会了提取“特征”(比如: 第一个神经元学会了判断“是不是 (1,0)”, 第二个学会了判断“是不是 (0,1)”), 最后输出层只需要做简单的加法就能得到正确答案。

代码实现-PyTorch

Python

```

1  import torch
2  import torch.nn as nn
3  import torch.optim as optim
4
5  # 1. 准备数据 (Data Preparation)
6  # PyTorch 喜欢浮点型数据, 且通常是 (batch_size, features) 的形状
7  X = torch.tensor([[0, 0],
8                   [0, 1],
9                   [1, 0],
10                  [1, 1]], dtype=torch.float32)
11  y = torch.tensor([[0],
12                  [1],

```

```

13             [1],
14             [0]], dtype=torch.float32)
15
16 # 2. 定义模型 (Model Definition)
17 # 继承 nn.Module 是 PyTorch 的标准写法
18 class MLP(nn.Module):
19     def __init__(self):
20         super(MLP, self).__init__()
21         # 定义网络层
22         # 输入层(2) -> 隐藏层(2) -> 输出层(1)
23         self.hidden_layer = nn.Linear(2, 2) # 输入2个特征，输出2个特征（隐藏层2
        个神经元）
24         self.output_layer = nn.Linear(2, 1) # 输入2个特征，输出1个预测值
25         self.activation = nn.Sigmoid()      # 激活函数
26
27     def forward(self, x):
28         # 定义数据流方向
29         x = self.activation(self.hidden_layer(x)) # 隐藏层计算 + 激活
30         x = self.activation(self.output_layer(x)) # 输出层计算 + 激活
31         return x
32
33 # 3. 实例化模型、定义损失函数和优化器
34 model = MLP()
35 criterion = nn.MSELoss() # 均方误差损失 (Mean Squared Error)
36 optimizer = optim.SGD(model.parameters(), lr=0.5) # 随机梯度下降
37
38 # 4. 训练循环 (Training Loop)
39 print("开始训练 PyTorch MLP...")
40 epochs = 5000
41 for epoch in range(epochs):
42     # 前向传播：计算预测值
43     predictions = model(X)
44
45     # 计算损失
46     loss = criterion(predictions, y)
47
48     # 反向传播：清零梯度 -> 反向传播 -> 更新权重
49     optimizer.zero_grad() # 清空上一轮的梯度，防止累加
50     loss.backward()       # 自动计算所有参数的梯度（核心魔法！）
51     optimizer.step()      # 根据梯度更新参数
52
53     if epoch % 1000 == 0:
54         print(f"Epoch [{epoch}/{epochs}], Loss: {loss.item():.4f}")
55
56 # 5. 测试模型
57 print("\n训练完成，预测结果：")
58 with torch.no_grad(): # 测试时不需要计算梯度
59     test_preds = model(X)

```

```

60     # 将概率转换为 0 或 1
61     predicted_classes = (test_preds > 0.5).float()
62     print(predicted_classes)

```

A. 告别“手算梯度” (The End of Manual Gradients)

- **以前 (NumPy):** 你需要手动推导并实现 `sigmoid_derivative`, 还要小心翼翼地用链式法则计算 `dw1`, `dw2`。一旦网络深了, 这几乎是不可能完成的任务。
- **现在 (PyTorch):** 只需要一行 `loss.backward()`。PyTorch 会动态构建计算图, 自动帮你算出所有参数 (`model.parameters()`) 的梯度。

B. 模块化封装 (Modular Design)

- `nn.Linear`: 帮你封装了 $wx + b$ 的计算, 不用手动写 `np.dot`。
- `nn.Sigmoid`: 封装了激活函数。
- `nn.MSELoss`: 封装了损失函数的计算。

C. 优化器接管更新 (Optimizer)

- **以前:** 你需要写 `self.W1 += ... * self.lr`。
- **现在:** `optimizer.step()` 会自动根据刚才算出的梯度 (`grad`) 和学习率去更新参数。

特性	感知机 (Perceptron)	多层感知机 (MLP)
结构	单层 (输入 -> 输出)	多层 (输入 -> 隐藏层 -> 输出)
激活函数	阶跃函数 (Step)	非线性函数 (Sigmoid, ReLU, Tanh)
学习能力	仅限线性可分问题	非线性问题 (如 XOR)
训练算法	简单的权重加减	反向传播 (Backpropagation) + 梯度下降
地位	神经网络的开端	深度学习的基石

所以: 只要层数够深、参数够多, 神经网络理论上可以拟合任意复杂的函数。

这也是为什么后来人们开始疯狂堆叠层数, 从而诞生了 AlexNet、ResNet 等现代深度学习巨兽的原因。

3.3 激活函数

3.3.1 激活函数

如果说**损失函数**是机器的“纠错指南针”，**多层感知机（MLP）**是机器的“大脑结构”，那么**激活函数（Activation Function）**就是神经元的“灵魂开关”。

没有激活函数，无论你堆叠多少层神经网络，整个模型本质上仍然只是一个**线性变换**（Linear Transformation）。

就像无论你把橡皮筋拉长多少倍，它依然是直的；只有加上“弯折”（非线性），才能拟合复杂的曲线。

是什么

赋予神经网络**非线性能力**的数学函数。如果没有它，无论网络有多少层，最终都等价于一个线性回归。

3.3.2 为什么需要激活函数？

假设一个没有激活函数的两层神经网络： $Output = W_2(W_1X + b_1) + b_2$

展开后： $Output = (W_2W_1)X + (W_2b_1 + b_2)$

令 $W' = W_2W_1$ ， $b' = W_2b_1 + b_2$ ，你会发现： $Output = W'X + b'$

这依然是一个线性方程，无论层数多深，没有激活函数的网络 = 单层感知机。

激活函数的作用：在每一层线性计算之后，**强行注入非线性**，让模型学会“弯曲”的决策边界，从而解决像 XOR 这样的复杂问题。

3.3.3 常见激活函数

名称	公式	图像特征	优点	缺点
Sigmoid (经典入门)	$\sigma(x) = \frac{1}{1+e^{-x}}$	呈S型，将输入压缩到 (0, 1) 之间。	1. 输出范围固定，适合表示概率 2. 平滑可导	1. 梯度消失： 当 x 很大或很小时，梯度接近0，导致深层网络无法训练。 2. 非零中心： 导致梯度更新方向不一致，收敛慢。

Tanh (双曲正切)	$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	呈S型，将输入压缩到 (-1, 1) 之间。	1. 零中心 (Zero-centered), 收敛比 Sigmoid 快 2. 梯度比 Sigmoid 强	1. 依然没有完全解决 梯度消失 问题。
ReLU (Rectified Linear Unit)	$R(x) = \max(0, x)$	左侧为0，右侧是一条直线。	1. 计算极快 (只需一个阈值判断) 2. 收敛极快 (梯度不会饱和) 3. 缓解了梯度消失问题 (在正区间)	1. Dead ReLU 问题 : 如果输入是负数，梯度永远为0，神经元可能永久死亡，不再更新。
Leaky ReLU (ReLU的改进)	$f(x) = \max(0.01x, x)$	类似 ReLU，但负数区域有一个微小的斜率 (如0.01)。	解决了 Dead ReLU 问题，负数区域也有微小梯度	性能通常略逊于 ReLU，但在某些特定任务上表现更好。
Softmax (多分类专用)	$S_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$	将一组数值转换为 概率分布 (总和为1)。	专为多分类输出层设计，能清晰表示属于各类别的概率	一般只用在最后一层，不适合隐藏层。

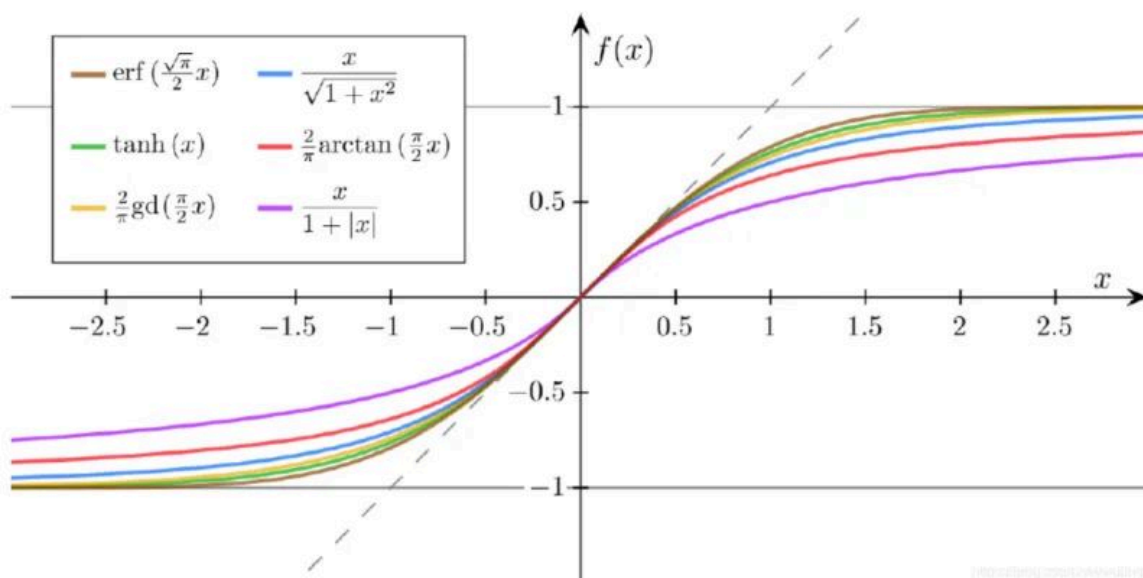
常见类型

- **Sigmoid**: 把值压缩到(0,1)，适合表示概率，但容易梯度消失。
- **Tanh**: 把值压缩到(-1,1)，比Sigmoid收敛快一点。
- **ReLU (Rectified Linear Unit)**: $f(x) = \max(0, x)$ 。目前最常用，计算快，解决了梯度消失问题 (正区间)。

3.3.4 梯度消失与 Dead ReLU

梯度消失 (Vanishing Gradient)

Sigmoid functions



观察 **Sigmoid** 的图像，当 x 很大（比如 10）或很小（比如 -10）时，曲线变得非常平。这意味着导数（斜率）接近于 0。

- **后果：**在反向传播中，梯度会连乘。如果每一层都是接近0的梯度，传到前面几层时，梯度就变成了 0，前面的参数就无法更新了。这就是为什么现在很少用 Sigmoid 做隐藏层的原因。

Dead ReLU 问题

ReLU 看起来很完美，但它有个致命弱点：如果某个神经元的输入总是负数，那么它的输出永远是 0，梯度也是 0。

- **后果：**这个神经元就“死”了，再也不会对任何数据有反应。这通常是因为学习率设置得太高导致的。

3.3.5 选择方案

- **隐藏层 (Hidden Layers)：**默认使用 **ReLU**。它是目前的行业标准，速度快，效果好。如果遇到 Dead ReLU，可以尝试 **Leaky ReLU** 或 **ELU**。
- **二分类输出层 (Binary Classification)：**使用 **Sigmoid**。因为它能把输出映射到 (0, 1)，正好代表概率。
- **多分类输出层 (Multi-class Classification)：**使用 **Softmax**。它能把多个输出值转化为概率分布，且总和为 1。

总结一句话：激活函数是让神经网络“活”起来的关键。

从 Sigmoid 到 ReLU 的演变，其实就是一场**对抗梯度消失、追求极致效率**的革命。

3.3.6 代码实现

如果把神经元比作水龙头，激活函数就是阀门。Sigmoid 是那种精细调节阀，水流很小；ReLU 则是“要么不开，要么全开”的开关阀，简单粗暴效率高。

Python

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # 定义激活函数
5 def sigmoid(x):
6     return 1 / (1 + np.exp(-x))
7
8 def tanh(x):
9     return np.tanh(x)
10
11 def relu(x):
12     return np.maximum(0, x)
13
14 def leaky_relu(x, alpha=0.01):
15     return np.where(x > 0, x, x * alpha)
16
17 # 生成数据
18 x = np.linspace(-5, 5, 100)
19
20 # 绘图
21 plt.figure(figsize=(12, 8))
22
23 plt.subplot(2, 2, 1)
24 plt.plot(x, sigmoid(x), color='blue')
25 plt.title("Sigmoid")
26 plt.grid(True)
27
28 plt.subplot(2, 2, 2)
29 plt.plot(x, tanh(x), color='green')
30 plt.title("Tanh")
31 plt.grid(True)
32
33 plt.subplot(2, 2, 3)
34 plt.plot(x, relu(x), color='red')
35 plt.title("ReLU")
36 plt.grid(True)
37
38 plt.subplot(2, 2, 4)
39 plt.plot(x, leaky_relu(x), color='purple')
40 plt.title("Leaky ReLU")
41 plt.grid(True)
42
```

```
43 plt.tight_layout()
44 plt.show()
```

3.3.7 梯度传播 (Gradient Flow)

激活函数如果不给力，会导致**梯度消失**，这背后的数学本质：**激活函数的导数到底长什么样？它们是如何在反向传播中“扼杀”梯度的？**

反向传播使用链式法则。假设某一层的线性计算是 $z = Wx + b$ ，经过激活函数后得到 $a = \sigma(z)$ 。

当我们求损失 L 对参数 W 的偏导数时，公式是：

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial W}$$

请注意中间的这项： $\frac{\partial a}{\partial z} = \sigma'(z)$ ，这就是**激活函数的导数**。

它就像一个“水龙头”，直接决定了梯度能有多少流回到前一层。如果 $\sigma'(z)$ 很小，梯度就会被截断，这就是**梯度消失**的数学根源。

A. Sigmoid 的导数：优雅的“自带项”公式

原函数： $\sigma(x) = \frac{1}{1+e^{-x}}$

求导结果（利用除法求导法则和链式法则）： $\sigma'(x) = \frac{e^{-x}}{(1+e^{-x})^2}$

这个形式可以通过数学变形，用原函数 $\sigma(x)$ 表示出来：

$$\sigma'(x) = \sigma(x) \cdot (1 - \sigma(x))$$

Sigmoid 的导数最大值出现在 $x = 0$ 处，最大值为 **0.25**。

这意味着，每经过一个 Sigmoid 层，梯度至少要被压缩 4 倍。如果有 5 层，梯度就会变成原来的 $(0.25)^5 \approx 0.1\%$ ，深层网络根本学不动！

B. Tanh 的导数：Sigmoid 的“放大版”

原函数： $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$

导数公式： $\tanh'(x) = 1 - \tanh^2(x)$

结论：Tanh 的导数最大值是 1（在 $x = 0$ 处）。这比 Sigmoid 好了 4 倍！但在两端（ $|x|$ 很大时），导数依然会衰减到 0，所以依然无法完全解决梯度消失。

C. ReLU 的导数：简单粗暴的“开关”

原函数： $R(x) = \max(0, x)$

导数公式 (分段函数): $R'(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$

结论: 当 $x > 0$ 时, 导数为 1! 梯度可以**无损地**穿过这一层。这就是为什么现代深层网络 (如 ResNet、CNN) 偏爱 ReLU 的原因。

(代价就是 $x < 0$ 时梯度为 0, 导致了 *Dead ReLU* 问题)

Python

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # 定义原函数及其导数
5 def sigmoid(x):
6     return 1 / (1 + np.exp(-x))
7
8 def sigmoid_derivative(x):
9     s = sigmoid(x)
10    return s * (1 - s)
11
12 def tanh(x):
13    return np.tanh(x)
14
15 def tanh_derivative(x):
16    return 1 - tanh(x)**2
17
18 def relu(x):
19    return np.maximum(0, x)
20
21 def relu_derivative(x):
22    return np.where(x > 0, 1, 0)
23
24 # 生成数据
25 x = np.linspace(-5, 5, 100)
26
27 # 绘图
28 plt.figure(figsize=(12, 4))
29
30 # Sigmoid 导数
31 plt.subplot(1, 3, 1)
32 plt.plot(x, sigmoid_derivative(x), color='blue', label="Sigmoid
    Derivative")
33 plt.axhline(y=0.25, color='red', linestyle='--', label='Max gradient
    (0.25)')
34 plt.title("Sigmoid: 梯度持续衰减")
```

```

35 plt.grid(True)
36 plt.legend()
37
38 # Tanh 导数
39 plt.subplot(1, 3, 2)
40 plt.plot(x, tanh_derivative(x), color='green', label="Tanh Derivative")
41 plt.axhline(y=1, color='red', linestyle='--', label='Max gradient (1)')
42 plt.title("Tanh: 梯度有所改善")
43 plt.grid(True)
44 plt.legend()
45
46 # ReLU 导数
47 plt.subplot(1, 3, 3)
48 plt.plot(x, relu_derivative(x), color='purple', label="ReLU Derivative")
49 plt.title("ReLU: 梯度无损传递 (x>0)")
50 plt.grid(True)
51 plt.legend()
52
53 plt.tight_layout()
54 plt.show()

```

3.3.8. 终极串联：梯度传递的关键瓶颈

假设一个简单的 3 层网络（输入层 → 隐藏层 → 输出层），使用 Sigmoid 作为激活函数。

定义输出层的误差项为 $\delta^{(out)}$ ，隐藏层的误差项为 $\delta^{(hid)}$ 。

根据链式法则，隐藏层误差的计算公式为：

$$\delta^{(hid)} = \delta^{(out)} \cdot W^{(out)} \cdot \sigma'(z^{(hid)})$$

其中：

- $W^{(out)}$ ：**隐藏层到输出层之间的权重矩阵**。如果初始化不当（**例如采用过小的随机数**），其**元素数值偏小会导致连乘后的梯度值进一步缩小**。
- $\sigma'(z^{(hid)})$ ：Sigmoid **函数在其输入 $z^{(hid)}$ 处的导数值**。根据公式 $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ ，**该导数最大值仅为 0.25（当 $z = 0$ 时）**。

在深度网络中，每一层的误差项都以此形式反向传播，导致梯度信号在多层连乘后呈指数级衰减，即**梯度消失问题** (Vanishing Gradient Problem)，这是制约深层网络训练的关键瓶颈之一。

ReLU 激活函数的出现，将其导数 $f'(z)$ 设为 1（当 $z > 0$ 时），实质上是移除了多层链式法则中每一层的缩减弱化因子，犹如打通了梯度流动的“高速通道”，使得训练真正意义上的深层神经网络成为可能。

激活函数的选择不仅决定了神经元的非线性建模能力 (activation shape)，其导数 $f'(z)$ 的幅值 (magnitude) 更是直接决定了反向传播过程中梯度信号的衰减或放大程度，从而掌控了整个网络训练的收敛命脉。

3.4 反向传播算法

3.4.1 反向传播算法

反向传播算法 (Backpropagation) 是训练神经网络的“心脏”。

核心逻辑就是**链式法则 (Chain Rule)** 的反复应用：计算损失函数对每一个权重的偏导数（梯度），然后交给优化器去更新。

是什么

神经网络的“学习机制”，通过计算**梯度**（导数），把误差从输出层一层层传回输入层，告诉每一层的参数该怎么调整才能减少误差。

步骤	方向	操作	核心公式 (LaTeX)
Forward	→	计算线性组合与激活	$\mathbf{z}^{(h)} = \mathbf{W}^{(h)}\mathbf{x} + \mathbf{b}^{(h)}, \quad \mathbf{a}^{(h)} = \sigma(\mathbf{z}^{(h)})$
Loss	→	计算预测误差	$\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$ (MSE)
Output Delta	←	计算输出层误差	$\delta^{(o)} = (\hat{y} - y) \odot \sigma'(\mathbf{z}^{(o)})$
Backward	←	误差传回隐藏层	$\delta^{(h)} = ((\mathbf{W}^{(o)})^\top \delta^{(o)}) \odot \sigma'(\mathbf{z}^{(h)})$
Gradients	←	计算参数梯度	$\nabla_{\mathbf{W}^{(o)}} = \mathbf{a}^{(h)}(\delta^{(o)})^\top, \quad \nabla_{\mathbf{W}^{(h)}} = \mathbf{x}(\delta^{(h)})^\top$
Update	←	更新权重	$\mathbf{W} \leftarrow \mathbf{W} - \eta \nabla_{\mathbf{W}}$

3.4.2 推导过程

1. 符号定义与前向传播

设有单隐藏层 MLP：

- 输入向量： $\mathbf{x}$
- 隐藏层线性组合： $\mathbf{z}^{(h)} = \mathbf{W}^{(h)}\mathbf{x} + \mathbf{b}^{(h)}$

- 隐藏层激活输出： $\mathbf{a}^{(h)} = \sigma(\mathbf{z}^{(h)})$ （ σ 为激活函数，如 Sigmoid/ReLU）
- 输出层线性组合： $z^{(o)} = \mathbf{W}^{(o)} \mathbf{a}^{(h)} + b^{(o)}$ （此处输出为标量，简化推导）
- 最终预测： $\hat{y} = a^{(o)} = \sigma(z^{(o)})$
- 损失函数（均方误差, MSE）： $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$

2. 反向传播计算梯度

a. 输出层参数梯度：

- 损失对输出层激活的梯度： $\frac{\partial \mathcal{L}}{\partial a^{(o)}} = \hat{y} - y$
- 激活函数对线性输入的梯度： $\frac{\partial a^{(o)}}{\partial z^{(o)}} = \sigma'(z^{(o)})$
- 损失对输出层线性输入的梯度（误差项）： $\delta^{(o)} = \frac{\partial \mathcal{L}}{\partial z^{(o)}} = (\hat{y} - y) \cdot \sigma'(z^{(o)})$
- 利用链式法则，得到参数梯度：

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(o)}} = \delta^{(o)} (\mathbf{a}^{(h)})^\top, \quad \frac{\partial \mathcal{L}}{\partial b^{(o)}} = \delta^{(o)}$$

b. 隐藏层参数梯度：

- 损失对隐藏层输出的梯度： $\frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(h)}} = (\mathbf{W}^{(o)})^\top \delta^{(o)}$
- 隐藏层激活函数对线性输入的梯度： $\frac{\partial \mathbf{a}^{(h)}}{\partial \mathbf{z}^{(h)}} = \text{diag}(\sigma'(\mathbf{z}^{(h)}))$ ，实际操作中使用逐元素乘。
- 损失对隐藏层线性输入的梯度（误差项）： $\delta^{(h)} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(h)}} = ((\mathbf{W}^{(o)})^\top \delta^{(o)}) \odot \sigma'(\mathbf{z}^{(h)})$ ，其中 $\odot$ 表示逐元素乘 (Hadamard积)。
- 最终得到参数梯度：

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(h)}} = \delta^{(h)} \mathbf{x}^\top, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(h)}} = \delta^{(h)}$$

3. 参数更新使用梯度下降法：

$\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{W}}$ ， $\mathbf{b} \leftarrow \mathbf{b} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{b}}$ ，其中 η 为学习率。

a. 输入层接收数据：将原始数据（图像、文本）转换为数值向量。

b. 隐藏层计算：

- 线性加权组合： $\mathbf{z} = \mathbf{W} \cdot \mathbf{x} + \mathbf{b}$ ， $\mathbf{z}$ 为预激活值。
- 非线性激活： $\mathbf{a} = f(\mathbf{z})$ ，激活函数 f （如ReLU, Sigmoid）引入非线性，使网络能够学习复杂模式，将数据映射到高维空间以便可分。

- c. **输出层生成预测**：最终层经线性变换和激活函数（如Softmax）处理，生成预测结果 $\hat{y}$ 。
- d. **损失计算**：将预测结果 $\hat{y}$ 与真实标签比较，计算损失函数值 $\mathcal{L}$ （如均方误差MSE或交叉熵），衡量模型准确性。

实现过程

1. **前向传播**：算出预测值，计算损失（Loss）。
2. **链式法则求导**：利用微积分的链式法则，计算损失函数相对于每个权重参数的梯度。
3. **误差回传**：从最后一层开始，计算每一层对总误差的“贡献度”，并修正权重。

3.4.3 代码实现

就像你射箭。第一箭射偏了（前向传播，算出Loss）。

分析：是因为手抖（层3的问题）？还是因为姿势不对（层2的问题）？还是因为风向（层1的问题）？反向传播就是从结果倒推，精确地算出每一步该调整多少角度。

Python

```
1 import numpy as np
2
3 # 1. 定义激活函数及其导数
4 def sigmoid(x):
5     return 1 / (1 + np.exp(-x))
6
7 def sigmoid_derivative(a):
8     # 注意：这里传入的是 sigmoid 的输出值 a，而不是 x
9     # 因为 sigmoid 的导数可以表示为 a*(1-a)
10    return a * (1 - a)
11
12 # 2. 初始化参数
13 x = 1.0          # 输入
14 t = 0.8          # 目标值 (Target)
15 w1, b1 = 0.5, 0.1
16 w2, b2 = 0.3, 0.2
17 lr = 0.1        # 学习率
18
19 print("--- 前向传播 (Forward Pass) ---")
20 # 3. 前向传播
21 z1 = w1 * x + b1
22 h = sigmoid(z1)
23 z2 = w2 * h + b2
24 y = sigmoid(z2)
25 loss = 0.5 * (y - t) ** 2
```



```

26
27 print(f"输入: x={x}, 目标: t={t}")
28 print(f"隐藏层: z1={z1:.4f}, h={h:.4f}")
29 print(f"输出层: z2={z2:.4f}, y={y:.4f}")
30 print(f"损失值: L={loss:.4f}")
31
32 print("\n--- 反向传播 (Backward Pass) ---")
33 # 4. 反向传播 (链式法则)
34
35 # --- 输出层梯度 ---
36 # 误差项 delta2 = (y - t) * sigmoid'(y)
37 delta2 = (y - t) * sigmoid_derivative(y)
38 # 权重梯度 = delta2 * 输入(h)
39 dw2 = delta2 * h
40 db2 = delta2 # 偏置的梯度就是 delta 本身
41
42 print(f"输出层误差 delta2: {delta2:.4f}")
43 print(f"输出层梯度 dw2: {dw2:.4f}, db2: {db2:.4f}")
44
45 # --- 隐藏层梯度 ---
46 # 误差项 delta1 = (delta2 * w2) * sigmoid'(h)
47 delta1 = (delta2 * w2) * sigmoid_derivative(h)
48 # 权重梯度 = delta1 * 输入(x)
49 dw1 = delta1 * x
50 db1 = delta1
51
52 print(f"隐藏层误差 delta1: {delta1:.4f}")
53 print(f"隐藏层梯度 dw1: {dw1:.4f}, db1: {db1:.4f}")
54
55 print("\n--- 参数更新 (Update) ---")
56 # 5. 更新参数
57 w1_new = w1 - lr * dw1
58 b1_new = b1 - lr * db1
59 w2_new = w2 - lr * dw2
60 b2_new = b2 - lr * db2
61
62 print(f"权重更新: w1 ({w1:.4f} -> {w1_new:.4f}), w2 ({w2:.4f} -> {w2_new:.4f})")
63 print(f"偏置更新: b1 ({b1:.4f} -> {b1_new:.4f}), b2 ({b2:.4f} -> {b2_new:.4f})")

```

人工智能作为计算机科学的重要分支，旨在让计算机具备模拟人类智能与自主解决问题的能力。

3.5 优化器

3.5.1 优化器

优化器是神经网络训练的核心引擎，负责在每一次反向传播后，根据计算出的梯度更新网络的权重和偏置参数，以最小化损失函数，引导模型朝着最优解的方向迭代。

是什么

拿到反向传播算出的梯度后，**具体怎么更新权重**的规则。决定了模型学习的速度和稳定性。

定义与目标

优化器 (Optimizer) 是实现梯度下降算法的具体实施方案。其核心目标是高效、稳定地找到损失函数的全局最优解 (或一个足够好的局部最优解)，从而最小化模型预测值与真实标签之间的误差。

工作原理 (梯度下降)

所有优化器都基于梯度下降的核心理念：参数沿着损失函数梯度的反方向进行微小调整。基本公式为

$\theta_{t+1} = \theta_t - \eta \cdot \nabla_{\theta} L(\theta_t)$ ，其中 θ 为参数， η 是学习率， ∇L 是损失函数的梯度。学习率决定了每次更新的步长，是至关重要的超参数。

常见类型与演进

- 随机梯度下降 (SGD)**: 最基础的形式。每次迭代使用一个样本 (或一个小批次) 计算梯度并更新参数。优点是实现简单、内存占用小，但其更新方向噪声大，容易震荡，收敛速度慢且对学习率敏感。
- 带动量的SGD (SGD with Momentum)**: 在SGD基础上引入“动量”概念，模拟物理惯性。更新时不仅考虑当前梯度，还会累积历史梯度的指数加权平均。这有助于加速在一致方向上的收敛，抑制震荡，尤其在损失函数“沟壑”地形中表现更佳。
- 自适应学习率优化器**: 这类优化器为每个参数动态调整学习率。
 - **AdaGrad**: 根据参数的历史梯度平方和调整学习率，适合处理稀疏数据。但学习率会持续衰减，可能导致后期训练停滞。
 - **RMSprop**: 对AdaGrad的改进，使用指数衰减平均历史梯度平方，解决了学习率过早衰减的问题，在非平稳目标上表现稳定。
 - **Adam (Adaptive Moment Estimation)**: 目前最流行的优化器之一。它结合了动量 (一阶矩估计) 和RMSprop (二阶矩估计) 的优点，既考虑梯度方向，也自适应地调整每个参数的学习率。通常具有收敛快、超参数鲁棒性强等特点。

选择考量

- **任务与数据**：Adam通常是良好的默认选择，尤其对于大多数深度学习任务。对于稀疏特征或需要更强泛化能力的场景，SGD（或带动量）配合合适的学习率调度策略有时能达到更好的最终性能。
- **收敛速度 vs. 泛化能力**：自适应优化器（如Adam）通常收敛更快，而朴素的SGD有时能找到更平坦、泛化能力更强的极小值点。

常见类型

- **SGD (随机梯度下降)**：最基础，沿着梯度反方向走一步。容易震荡。
- **Momentum (动量)**：加入了惯性，像推球下山，让更新更平滑。
- **Adam**：目前的王者。它结合了动量和自适应学习率，基本上“傻瓜式”好用，不需要怎么调参。

3.5.2 实现过程

现代深度学习框架（如PyTorch、TensorFlow）已将主流优化器封装为标准模块。

实现过程包括：

1. **定义模型与损失函数**。
2. **实例化优化器**：指定优化器类型（如 `torch.optim.Adam`），并传入需要优化的模型参数，同时设置学习率等超参数。
3. **训练循环**：在每个训练批次（batch）中，依次执行：前向传播 -> 计算损失 -> 清空历史梯度（`.zero_grad()`） -> 反向传播（`.backward()`） -> 优化器更新参数（`.step()`）。

3.5.3 代码示例

使用PyTorch和Adam优化器的简化代码

反向传播告诉你“往哪走”，优化器决定“怎么走”。SGD是盲人摸象，走一步算一步；Adam就像一个开着导航且熟悉路况的老司机，知道哪里该加速，哪里该减速，最快到达目的地。

Python

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4
5 # 1. 定义简易模型
6 model = nn.Linear(in_features=10, out_features=1)
7 # 2. 定义损失函数（以均方误差为例）
8 criterion = nn.MSELoss()
9 # 3. 实例化Adam优化器，学习率设为0.001
```

```

10 optimizer = optim.Adam(model.parameters(), lr=0.001)
11
12 # 模拟训练数据
13 inputs = torch.randn(32, 10) # batch_size=32, feature_size=10
14 labels = torch.randn(32, 1)
15
16 # 4. 训练循环
17 model.train()
18 for epoch in range(100):
19     # 前向传播
20     outputs = model(inputs)
21     loss = criterion(outputs, labels)
22
23     # 反向传播与优化
24     optimizer.zero_grad() # 清空梯度缓存
25     loss.backward()        # 计算梯度
26     optimizer.step()       # 更新参数
27
28     if epoch % 10 == 0:
29         print(f'Epoch [{epoch}/100], Loss: {loss.item():.4f}')

```

3.6 正则化

3.6.1 正则化

正则化是一系列用于防止神经网络在训练数据上**过拟合**的技术集合。其核心目标是提高模型在未见过的测试数据上的**泛化能力**，使模型不仅“记住”训练集，更能“理解”数据背后的普遍规律。

正则化通过在模型训练过程中引入额外的约束或噪声，限制模型的复杂度，从而避免模型过度适应训练数据中的噪声和无关细节。一个过拟合的模型通常具有较低的训练误差，但测试误差很高。

正则化旨在平衡**拟合能力**与**泛化能力**。

1. 是什么

为了防止模型“死记硬背”（过拟合），强制让模型变得更简单、更泛化的手段。

主要方法及其作用机制

1. 参数范数惩罚（L1/L2正则化）

- 原理：**直接在损失函数 L 后增加一个关于模型权重 θ 的惩罚项 $\Omega(\theta)$ ，形成新的目标函数 $\tilde{L} = L + \lambda\Omega(\theta)$ ，其中 λ 是控制惩罚强度的正则化系数。

- **L2正则化 (权重衰减)**: 惩罚项为 $\Omega(\theta) = \frac{1}{2} \|\theta\|_2^2$ 。它倾向于让所有权重衰减趋近于0, 但通常不等于0, 从而得到一个权重较小且分布更平滑的模型, 提升稳定性。在物理上可理解为约束参数值在一个小球内。
- **L1正则化**: 惩罚项为 $\Omega(\theta) = \|\theta\|_1$ 。它能产生**稀疏解**, 即将大量的权重压缩至0, 相当于自动进行了特征选择, 使模型更简单、更具解释性。

2. Dropout

- **原理**: 在训练阶段, 以概率 p (如0.5) 随机地将网络中一部分神经元的输出值置零 (暂时“丢弃”)。这强迫网络不依赖任何单个神经元或少数路径, 必须学习到更鲁棒、更具冗余性的特征表示。
- **作用机制**: Dropout可以被视为一种在大量“子网络”上进行模型平均的近似。在测试阶段, 所有神经元被激活, 但其输出值会乘以 $1 - p$ (或权重除以 $1 - p$), 以保持期望输出的一致性。

1. 权重衰减 (Weight Decay)

- 这是实现L2正则化的一种具体操作形式。在每次参数更新时, 除了减去梯度项, 还会主动将当前的权重值乘以一个略小于1的衰减因子 $(1 - \lambda_{wd})$ 。在许多优化器 (如AdamW) 中, 权重衰减与L2正则化在数学上是不同的。

3. 其他方法

- **数据增强**: 通过人工变形 (如翻转、裁剪、加噪等) 扩充训练数据, 是视觉任务中最有效的正则化手段之一。
- **早停 (Early Stopping)**: 监控模型在验证集上的性能, 当性能不再提升时即停止训练, 防止模型在训练集上过度优化。

常见方法

L1/L2 正则化: 在损失函数里加一项“惩罚”, 权重越大, 惩罚越重, 强迫模型少用没用的特征。

Dropout: 在训练过程中, **随机丢弃**一部分神经元 (把它们设为0)。这就像是在训练一支军队, 每次打仗随机让一部分士兵休假, 逼剩下的士兵学会独立作战, 提高整体鲁棒性。

3.6.2 实现过程

在框架中集成正则化方法非常便捷:

- **L1/L2正则化**: 通常在定义优化器时通过 `weight_decay` 参数 (对应L2) 或自定义损失函数实现。

- **Dropout:** 使用网络层模块（如 `nn.Dropout`）直接插入到网络架构中。它有 `train/eval` 两种模式，框架会自动处理训练与推断时的行为切换。
- **早停:** 需要在训练循环中手动实现验证评估与停止判断的逻辑。

3.6.3 代码示例

考试前你背下了所有课后习题答案（过拟合）。

正则化就是老师出的“变形题”。

L1/L2 是让你少背答案多理解原理；

Dropout 是让你闭着眼睛复习，强迫你靠逻辑推导而不是视觉记忆。

Python

```
1 import torch.nn as nn
2 import torch.optim as optim
3
4 # 定义带Dropout层的简单全连接网络
5 class SimpleNN(nn.Module):
6     def __init__(self, input_dim=20, hidden_dim=64, output_dim=1,
7                 dropout_rate=0.5):
8         super().__init__()
9         self.layer1 = nn.Linear(input_dim, hidden_dim)
10        # 在激活函数后加入Dropout层
11        self.dropout = nn.Dropout(p=dropout_rate)
12        self.layer2 = nn.Linear(hidden_dim, output_dim)
13        self.relu = nn.ReLU()
14
15    def forward(self, x):
16        x = self.relu(self.layer1(x))
17        x = self.dropout(x) # 训练时随机丢弃一部分激活值
18        x = self.layer2(x)
19        return x
20
21 model = SimpleNN()
22 criterion = nn.MSELoss()
23 # 在Adam优化器中设置 weight_decay 参数实现L2正则化（此处 $\lambda=0.01$ ）
24 optimizer = optim.Adam(model.parameters(), lr=0.001, weight_decay=0.01)
25
26 # 注意：在模型评估（推理）前，务必切换到 `eval` 模式，Dropout层会失效。
27 model.eval()
28 # 在训练前，切换回 `train` 模式。
29 model.train()
```

3.7 总结

神经网络的工作流

1. **数据**进入 **MLP**。
2. 经过 **激活函数** 变得非线性。
3. 算出结果，计算 **Loss**。
4. **反向传播** 算出梯度。
5. **优化器** 根据梯度更新权重。
6. 为了防止过拟合，加入 **Dropout** 或 **L2正则化**。